



TRABALHO DE ARQUITETURA E ORGANIZAÇÃO

PROJETO

SIMULADOR MIPS

GABRIEL MOLINA DE LIMA - RA: 2208423

GUILHERME MOREIRA LIMA FURLANETO - RA: 2207192

GUSTAVO GEOVANE TAMIÃO DE SOUZA - RA: 2271990

PEDRO MIAN PARRA - RA: 2207249

APUCARANA

2021



SUMÁRIO

INTRODUÇÃO	2
MATERIAL E MÉTODOS	2
MATERIAL	2
METODOLOGIA	3
Siglas e Subseção Terciária	4
RESULTADOS E DISCUSSÃO	4
CONSIDERAÇÕES FINAIS	5
REFERÊNCIAS	5



1. INTRODUÇÃO

Este projeto tem como objetivo criar uma versão do processador MIPS de 32 bits, capaz de ler e executar instruções binárias codificadas. O simulador foi desenvolvido utilizando a linguagem C, onde a manipulação de bits é feita com maior facilidade a outras linguagens de alto nível.

O simulador possui uma estrutura básica em termos de entrada e saída de dados. As informações dos arquivos devem ser passadas por parâmetros *arg C*, *arg V*. É necessário fornecer um arquivo binário contendo as instruções que serão realizadas, onde dois arquivos de texto serão gerados: um arquivo contendo um relatório de execução de todas as instruções (contendo instrução realizada, conteúdo dos registradores utilizados por essas instruções e conteúdo da memória); e outro arquivo de texto contendo o conteúdo de todos os registradores após a execução do simulador.

Para executar o simulador, primeiramente é necessário compilar o arquivo *.c*, depois de compilado, é necessário executar o arquivo passando os 3 parâmetros necessários: o arquivo de entrada binário; o arquivo de texto para conter o relatório das operações; o arquivo de texto para conter o conteúdo final dos registradores. A Figura 1 demonstra a execução do simulador pelo terminal.

Figura 1 – Exemplo de execução do simulador

```
C:\Users\GMLF\Downloads\projeto_aoc-main\projeto_aoc-main>mips input1.bin op_saida.txt reg_saida.txt

Simulador MIPS
1 - Rodar o programa inteiro
2 - Rodar passo a passo
Sua escolha: 1

Printando um valor inteiro...
55

Saindo do programa...

Finalizado!
O arquivo com todas as instrucoes realizadas, bem como o arquivo com o estado final de cada registrador
foi gerado no diretorio do programa
```

Fonte: Autoria própria

Após executado o comando é pedido se o programa deve ser rodado de uma vez ou passo a passo. Se rodado de uma vez todas as chamadas de sistema serão



mostradas e o arquivo contendo o relatório das operações e dos registradores será gerado. Se rodado passo a passo será necessário pressionar a tecla *enter* para rodar a próxima instrução, e os dados da instrução executada será mostrada no terminal, bem como gravado no arquivo de texto.

O simulador foi construído em dois arquivos, um arquivo cabeçalho contendo o código das operações (instruções) e outro contendo o restante do simulador (arquivo principal).

2. DESENVOLVIMENTO

As instruções implementadas podem ser observadas na Tabela 1. Esta tabela relaciona cada instrução com o seu código, ou seja, o comando utilizado para que esta instrução pudesse ser realizada.

Tabela 1 – Operações implementadas no simulador e suas ações

Operação	Descrição	Ação
ADD rd,rs,rt	Soma entre 2 registradores	rd=rs+rt
ADDI rt,rs,imm	Soma entre registrador e valor imediato	rt=rs+imm
ADDU rd,rs,rt	Soma entre 2 registradores (sem sinal)	rd=rs+rt
ADDIU rt,rs,imm	Soma entre registrador e valor imediato (sem sinal)	rt=rs+imm
SUB rd,rs,rt	Subtração entre 2 registradores	rd=rs-rt
MULT rs,rt	Multiplicação entre 2 registradores	HI,LO=rs*rt
DIV rs,rt	Divisão entre 2 registradores	HI=rs%rt; LO=rs/rt
AND rd,rs,rt	And lógico entre 2 registradores	rd=rs&rt
OR rd,rs,rt	Or lógico entre 2 registradores	rd=rs rt
XOR rd,rs,rt	Or exclusivo entre 2 registradores	rd=rs^rt
NOR rd,rs,rt	Nor lógico entre 2 registradores	rd=~(rs rt)
SLT rd,rs,rt	Atribui se menor que	rd=rs<rt
SLL rd,rt,sa	Deslocamento a esquerda entre 2 registradores	rd=rt<<sa
SRL rd,rt,sa	Deslocamento a direita entre 2 registradores	rd=rt>>sa
ANDI rt,rs,imm	And lógico entre registrador e valor imediato	rt=rs&imm
ORI rt,rs,imm	Or lógico entre registrador e valor imediato	rt=rs imm
XORI rt,rs,imm	Or exclusivo entre registrador e valor imediato	rt=rs^imm
SYSCALL	Chamada de sistema	-



LW rt,offset(rs)	Dados da memória para registrador	$rt = *(int*)(offset + rs)$
LB rt,offset(rs)	Byte da memória para registrador	$rt = *(char*)(offset + rs)$
LBU rt,offset(rs)	Byte sem sinal da memória para registrador	$rt = *(Uchar*)(offset + rs)$
LUI rt,imm	?	$rt = imm \ll 16$
SW rt,offset(rs)	Dados do registrador para memória	$*(int*)(offset + rs) = rt$
SB rt,offset(rs)	Byte do registrador para memória	$*(char*)(offset + rs) = rt$
BEQ rs,rt,offset	Salta se for igual	$if(rs == rt) \text{ pc} += offset * 4$
BNE rs,rt,offset	Salta se não for igual	$if(rs != rt) \text{ pc} += offset * 4$
BLEZ rs,offset	Salta se menor ou igual a zero	$if(rs \leq 0) \text{ pc} += offset * 4$
BGTZ rs,offset	Salta se maior que zero	$if(rs > 0) \text{ pc} += offset * 4$
SLTI rt,rs,imm	Atribui se menor que valor imediato	$rt = rs < imm$
SLTIU rt,rs,imm	Atribui se menor que valor imediato sem sinal	$rt = rs < imm$
J target	Salto no endereço destino	$pc = target$
JR rs	Salto no endereço do registrador	$pc = rs$
JAL target	Salto no endereço destino salvando atual	$r31 = pc; pc = target$

A operação *syscall* foi implementada para mostrar: um valor inteiro, um caractere e um *string* na tela. Também foram implementadas funções para ler: um valor inteiro, um caractere e um *string* do teclado e sair do programa.

Para que as instruções pudessem ser realizadas, foi necessário implementar um banco de registradores para englobar os 34 registradores do *MIPS* (incluindo os registradores *HI* e *LO*). Este banco foi criado utilizando um vetor de inteiros de 34 posições, onde o índice do vetor seria código do registrador e o conteúdo deste índice o valor armazenado pelo registrador.

Também foi implementada uma memória para simular a pilha, por meio de um vetor de inteiros que pode crescer para baixo (primeiro índice da memória é definido como o último índice do vetor) simulando o comportamento da memória do *MIPS*. É necessário reservar espaços na pilha somando valores ao registrador *\$sp*, para assim poder guardar valores na pilha, como no hardware real.

Foi usado um vetor de caracteres para armazenar *strings*, necessários para as funções de armazenar e mostrar na tela da operação *syscall*.

A execução do simulador segue a seguinte ordem: primeiro é realizada a contagem de operações contidas no arquivo de entrada; é alocado um vetor do



tamanho necessário para armazenar as operações; salva cada operação (valor inteiro de 32 bits) em um índice desse vetor.

Para cada operação, o simulador lê o valor do *opcode* (6 primeiros bits). Se este valor for zero é uma operação do tipo *r*, então deve ser lido o valor de *funct* para definir qual a operação da instrução. Os valores dos registradores também são lidos para realizar a operação. Então, para cada valor de *funct* diferente existe uma chamada para a função correspondente.

Se o valor do *opcode* for diferente de zero, a operação é ou do tipo *j* ou do tipo *i*. Neste caso, a operação é definida pelo *opcode*, sendo que para cada tipo de operação (*i* ou *j*) são retirados os valores necessários para execução da instrução.

Em cada operação rodada, o simulador escreve no relatório de execução a operação que está sendo executada no momento, o valor de *\$pc*, que aponta para o índice do vetor de operações, o valor dos registradores utilizados na operação e, em caso de operações que salvam valores na memória, o valor de cada índice utilizado da pilha implementada.

Caso a operação tenha um *opcode* ou *funct* não englobado pelo simulador, é indicado no arquivo de saída que esta operação é inválida. Caso tente ser realizados saltos que não englobam o escopo da simulação (para valores de *\$pc* menores que zero e valores superiores ao índice da última



instrução) também é indicado na saída que a operação é inválida, além de parar a execução do simulador.

O arquivo contendo o conteúdo final dos registradores é gerado no final da execução do programa. Neste arquivo é englobado o nome do registrador, o código do registrado e o valor armazenado por ele.

O programa pode ser executado de duas maneiras: execução instantânea de todas as operações, onde pode ser observada a execução das operações pelo arquivo de relatório; execução passo a passo, onde, em cada instrução executada é mostrado na tela o conteúdo que é escrito no arquivo de relatório. Ao executar o programa é pedido qual execução deve ser realizada.

3. RESULTADOS E DISCUSSÕES

Foram escritos 3 programas para testar o simulador. Os programas em código *assembly* podem ser observados abaixo.

Figura 2 – Programa para calcular soma recursivamente

```
main:
    addi $a0, $zero, 10      # atribui 10 para n (parametro da funcao)
    jal SOMA                 # vai para a função recursiva
    j OUTPUT                 # vai para a saída

SOMA:
    addi $sp, $sp, -8        # adiciona espaço para a stack
    sw $ra, 4($sp)           # guarda ra
    sw $a0, 0($sp)           # guarda n
    slt $t0, $a0, $zero      # if(n < 0), $t0 = 1
    beq $t0, $zero, ELSE     # if($t0 == 0) vai pro else
    add $v0, $zero, $zero     # return 0
    addi $sp, $sp, 8         # remove stack
    jr $ra                   # pula para $ra

ELSE:
    addi $a0, $a0, -1        # n = n - 1
    jal SOMA                 # soma com n - 1
    lw $a0, 0($sp)           # carrega n
    lw $ra, 4($sp)           # carrega ra
    addi $sp, $sp, 8         # remove stack
    add $v0, $a0, $v0        # return n + soma(n - 1)
    jr $ra                   # pula para $ra

OUTPUT:
    add $a0, $zero, $v0      # Coloca o retorno da função em $t0
    addi $v0, $zero, 1
    syscall
    add $v0, $zero, 10
    syscall
```

Figura 3 – Programa para calcular soma de uma subtração com funções

```
main:
    ....
    addi $a0, $zero, 5      # passa os valores para os argumentos
    ....
    addi $a1, $zero, 10     #
    ....
    addi $a2, $zero, 30     #
    ....
    addi $a3, $zero, 20     #
    ....
    jal F_SUM
    ....
    j OUTPUT

F_SUB:
    ....
    addi $sp, $sp, -4       # espaço para a subtracao
    ....
    sw $s0, 0($sp)          # aloca a subtracao
    ....
    sub $s0, $a2, $a3        # faz a subtracao
    ....
    add $v0, $s0, $zero      # coloca a subtracao em $v0
    ....
    addi $sp, $sp, 4        # remove da stack
    ....
    jr $ra                  # retorna para F_SUM
    ....

F_SUM:
    ....
    addi $sp, $sp, -8       # espaço para a soma e a $ra da main
    ....
    sw $ra, 4($sp)          # aloca o retorno para a main
    ....
    add $s0, $a0, $a1       # faz a soma de a + b
    ....
    sw $s0, 0($sp)          # aloca soma de a + b
    ....
    jal F_SUB               # faz a subtracao de c - d
    ....
    lw $s0, 0($sp)          # restaura o valor da soma de a + b
    ....
    lw $ra, 4($sp)          # restaura ra
    ....
    add $v1, $v0, $s0       # coloca a + b + (c - d) no retorno $v1
    ....
    addi $sp, $sp, 8        # remove da stack
    ....
    jr $ra                  # volta para a main
    ....

OUTPUT:
    ....
    addi $v0, $zero, 1      # Printa resultado
    ....
    add $a0, $zero, $v1     #
    ....
    syscall                 #
    ....
    addi $v0, $zero, 10     # Sai do programa
    ....
    syscall                 #
    ....
```

Fonte: Autoria própria

Figura 4 – Programa para calcular fatorial recursivamente

```
main:
    ....
    addi $a0, $zero, 5
    ....
    jal FACT
    ....
    j OUTPUT

FACT:
    ....
    addi $sp, $sp, -8       # gera espaço na stack para salvar os registradores necessários
    ....
    sw $ra, 4($sp)          # Endereço de retorno da função
    ....
    sw $a0, 0($sp)          # Aloca espaço para n
    ....
    slti $t0, $a0, 1        # n < 1
    ....
    beq $t0, $zero, L1      # se n >= 1 vai para L1
    ....
    addi $v0, $zero, 1      # return 1
    ....
    addi $sp, $sp, 8        # Retira 2 itens da stack
    ....
    jr $ra                  # Pula para o endereço de retorno
    ....

L1:
    ....
    addi $a0, $a0, -1       # chama fact com (n - 1)
    ....
    jal FACT
    ....
    lw $a0, 0($sp)          # restaura n
    ....
    lw $ra, 4($sp)          # restaura ra
    ....
    addi $sp, $sp, 8        # Retira 2 itens da stack
    ....
    mult $a0, $v0           # retorna n * fact(n - 1)
    ....
    mflr $v0
    ....
    jr $ra                  # retorna para quem chamou (main ou fact)
    ....

OUTPUT:
    ....
    add $t2, $zero, $v0
    ....
    addi $v0, $zero, 1
    ....
    add $a0, $zero, $t2
    ....
    syscall
    ....
    addi $v0, $zero, 10
    ....
    syscall
    ....
```

Fonte: Autoria própria



As instruções de cada código foram transformadas para valores binários de 32 bits e escritos em arquivos binários, para que pudessem ser executados no simulador. O código da Figura 2 calcula todos os valores 1 até 10 recursivamente, o código da Figura 3 calcula a expressão $5 + 10 + (30 - 20)$ e o código da Figura 4 calcula o valor de 5 fatorial.

Os 3 códigos foram executados, o resultado da execução do código da Figura 2 e as demais execuções podem ser observadas abaixo.

Figura 5 – Saída da execução do segundo programa

```
Prompt de Comando

C:\Users\GMLF\Downloads\projeto_aoc-main\projeto_aoc-main>mips input2.bin op_saida.txt reg_saida.txt

Simulador MIPS
1 - Rodar o programa inteiro
2 - Rodar passo a passo
Sua escolha: 1

Printando um valor inteiro...
25

Saindo do programa...

Finalizado!
O arquivo com todas as instrucoes realizadas, bem como o arquivo com o estado final de cada registrador
foi gerado no diretorio do programa
```

Fonte: Autoria própria

Figura 6 – Saída da execução do terceiro programa

```
C:\Users\GMLF\Downloads\projeto_aoc-main\projeto_aoc-main>mips input3.bin op_saida.txt reg_saida.txt

Simulador MIPS
1 - Rodar o programa inteiro
2 - Rodar passo a passo
Sua escolha: 1

Printando um valor inteiro...
120

Saindo do programa...

Finalizado!
O arquivo com todas as instrucoes realizadas, bem como o arquivo com o estado final de cada registrador
foi gerado no diretorio do programa

C:\Users\GMLF\Downloads\projeto_aoc-main\projeto_aoc-main>
```

Fonte: Autoria própria

O arquivo de relatório das operações e dos registradores da execução do código da Figura 4 pode ser observado abaixo.



Figura 7 – Trecho exemplo de relatório de operações

```
$pc = 0
Operacao: addi 4, 0, 5
Conteudo dos registradores:
  Registrador 4: 5
  Registrador 0: 0

$pc = 1
Operacao: jal 0x00100003
Conteudo dos registradores:
  Registrador 31: 1
  Novo $pc: 3

$pc = 3
Operacao: addi 29, 29, -8
Conteudo dos registradores:
  Registrador 29: 254
  Registrador 29: 254

$pc = 4
Operacao: sw 31, 4(29)
Conteudo dos registradores:
  Registrador 31: 1
  Registrador 29: 254
Conteudo da memoria:
  RAM[255]: 1

$pc = 5
Operacao: sw 4, 0(29)
Conteudo dos registradores:
  Registrador 4: 5
  Registrador 29: 254
Conteudo da memoria:
  RAM[254]: 5

$pc = 6
Operacao: slti 8, 4, 1
Conteudo dos registradores:
  Registrador 8: 0
  Registrador 4: 5
```

Fonte: Autoria própria



Figura 8 – Exemplo de relatório dos valores finais dos registradores

Nome	Numero	Valor
\$zero	0	0
\$at	1	0
\$v0	2	10
\$v1	3	0
\$a0	4	120
\$a1	5	0
\$a2	6	0
\$a3	7	0
\$t0	8	1
\$t1	9	0
\$t2	10	120
\$t3	11	0
\$t4	12	0
\$t5	13	0
\$t6	14	0
\$t7	15	0
\$s0	16	0
\$s1	17	0
\$s2	18	0
\$s3	19	0
\$s4	20	0
\$s5	21	0
\$s6	22	0
\$s7	23	0
\$t8	24	0
\$t9	25	0
\$k0	26	0
\$k1	27	0
\$gp	28	0
\$sp	29	256
\$fp	30	0
\$ra	31	1
pc		24
hi		0
lo		120

Fonte: Autoria própria

Não foi exibido o arquivo inteiro da Figura 7 devido à extensão causada pela quantidade de instruções realizadas.



4. CONSIDERAÇÕES FINAIS

Analisando os resultados demonstrados se conclui que o simulador consegue com êxito realizar as operações necessárias para finalizar os três programas utilizados como teste.

As instruções implementadas funcionam corretamente, como mostrado nos arquivos de saída.

Implementações futuras podem incluir a possibilidade de manipular número com ponto flutuante, visto que o simulador implementado apenas manipula valores inteiros e caracteres, bem como consegue armazenar e mostrar *strings*.

5. REFERÊNCIAS

- [1]SOUZA, F. **Bits em Linguagem C – Conceito e Aplicação**. 2015. Disponível em: <https://www.embarcados.com.br/bits-em-linguagem-c/>. Acesso em: 24 nov. 2021.
- [2]**Plasma - most MIPS I(TM) opcodes**. 2018. Disponível em: <https://opencores.org/projects/plasma/opcodes>. Acesso em: 24 nov. 2021.